Most of the logic of my ROM card is contained in four small PLDs, however I am sure that they could all be replaced by either discrete logic of a single large PLD. The PLDASM source files for the PLDs is included in this package.

The ROM images are stored in a 27C1024 64Kx16 EPROM with the lower 12 bits of the address selecting the individual words in a ROM image and the upper 4 bits selecting one of sixteen images. I only used this for expansion ROM images plugged into one of the RAM card connectors. If you wanted to connect to one of the ROM slots additional buffering on the inputs and outputs would be required. The ROM slots all use a 6V logic level, and are driven by a custom HP buffer chip. When I designed a ROM card for a HP 86B I was faced with the same issue, but found that CMOS logic such as the 74HC series was tolerant of 6V input, and to drive the 6V bus I used a 74HC buffer with its Vcc connected to the 6V power. This same design could be easily modified to contain the system ROMs and there would be enough space in a single 27C1024 for all the LPU and PPU images.

Since the memory space in the 9835 is divided into 32K 16 bit word blocks of which the even block numbers are occupied by RAM and the odd blocks, with the exception of block one, contain ROM. Block one has part of the first memory card faked into it to provide RAM for the virtual PPU function. The ROM blocks are further subdivided into eight 4K word segments each of which would be the memory space for a single ROM module.

Since each ROM segment is 4K words in size, the lower 12 bits of the address are latched by the 26V12 PLD named 12Bit. I could have used discrete latches, but by using a PLD it is all in one package instead of 2 or 3.

In my design a 74LS138 3-8 decoder is used to decode the upper three address bits to provide individual segment select lines which are combined with the decoded block select lines to select the desired image in the EPROM. Since the IDA bus uses negative logic the order of the outputs on the LS138 are reversed for example a value of zero in bits A12-A14 would be presented as 111 so output 7 (pin 7) would represent segment 0.

The PGSEL PLD decodes the Block select lines BSC0-6 and provides decoded outputs, in my case I generate outputs for octal block numbers 5, 13, and 21, however the logic is pretty straight forward and the PLD source file could be easily modified to accommodate other block numbers. Please note that signal lines BSC0-6 also use negative logic.

The ROMSEL PLD combines the output of the LS138 with the output of PGSEL PLD to select the desired image in the EPROM. It selects the image by setting the appropriate level for the upper 4 bits of the EPROM address. This PLD is also pretty straight forward and may need top be altered to suit your needs. This PLD also generates a SELECTED signal that is a logical OR of all the ROM selections this is used for some of the signal timing fictions of the last PLD MISC.

As the name implies the MISC PLD provides a couple different functions. The first is it inverts the ALE (-STM) signal from the memory bus, this signal is also used internally in the PLD. There are two signal lines which I did not name that act as handshakes in the ROM read cycle they are pins 32 and 40 on the memory bus. The cycle starts with the CPU putting out the address on the IDA bus and then ALE goes low to indicate that address is valid shortly after that the memory device signals it is ready by pulling down pin 40 the CPU responds by raising pin 32 and the ROM data appears on the IDA bus on the next rising edge of phase 2 clock. I found that I could operate on a short cycle so MISC latches the inverted ALE on the next rise of phase 2 and if the selected signal is active it pulls down pin 40.

The selected signal is inverted to drive the -CE signal low and when the pin 32 signal goes active it is anded with selected to drive the -OE on the EPROM and also enables the 74LS240 buffers.

I used LS240s for the output buffers since the IDA bus uses negative logic, using inverting buffers allowed me to program the EPROM contents as positive logic. I also used 2 gates of a LS125 to buffer the -OE and pin 40 signals. I probably do not need the buffer on the -OE, I had added it due to some early problems but the loading on that signal did not turn out to be the problem but the buffer remained. The pin 40 signal is a wired OR of all the memory devices and normally you would probably use an open collector device to drive it, but the arrangement that I have with the PLD output driving the buffers -enable line and the input grounded works just as well, pulling down the signal line when the pin 40 signal is to be activated and going to high impedance state when it is not.

I hope that this is sufficient explanation of how this works if you have questions I can be reached at phb.hfx@gmail.com.